The tools and techniques of R programming
Lesson-1
Introduction to R

By
Prof.Dr.A.B.Chowdhury,HOD,CA

Techno India University, West Bengal,India
For any query,Email-Id:hod.ca@technoindiaeducation.com

August 21, 2023

Learning OBJECTIVES OF THIS LESSON

This lesson aims to introduce the reader to the following concepts:

- **Concepts of R Environment**
- **Basic Advantages of R**
- **R Execution System**
- **R Character Set**
- **R Words**
- **Data Types in R**
- **R Variables,Constants and Literals**
- **R Operators**
- **Statements in R**
- **Installation of R**
- **R as a Displayer**
- **R as a Desk Calculator**
- **Objects in R**
- **R as a Scripting Language**

What is R?

R was created basically as a **scripting language** for statistical data manipulation,fast analysis of data and efficient graphical presentation.R is said to be a **scripting language** because it is an integrated suite of software environment that provides an interactive runtime platform to execute instructions directly without any compilation.The runtime environment of R is also capable of providing with additional functions, integrating complex systems, and communicating with other programming languages.

The incredibly comprehensive package of R can be utilized for Big Data Analytics, Machine Learning, signal processing, statistical tests and modeling, state-of-the-art static and dynamic graphical display, Data Mining, distributed computing and Object Oriented Programming apart from basic mathematical and statistical computations.It is an implementation of the S language which was developed at AT&T for statistical computations exclusively. R was created by **George Ross Ihaka and Robert Gentleman** under a **GNU project** at the University Of Auckland, New Zealand in 1991 and is currently developed by the R Development Core Team.The term GNU (pronounced as '**g-nu**') stands for a recursive acronym of **"GNU's Not Unix"**

R-the concepts to recognize it

The word **'gnu'** originates from the alternative name of a **wildebeest** which is a large dark antelope with a long head, a beard and mane, and a sloping back as shown in the figure below:

Figure: GNU-A wildebeest

Why Use R?

R is the first choice of professional statisticians and data scientists because it is **both inexpensive as well as beautiful.**
The following additional features of R lie behind its growing popularity:

- **It is usable in the Windows, Mac, and Linux operating systems which are used by most of the computer users**
- Although **R was created as a special-purpose scripting language** for data science and statistical operations, **it can also be used as a general-purpose programming language** like c, java etc. for developing any application.
- **The features of both object-oriented and functional programming languages are in-built in the language.**
- **Since the system saves data sets used between sessions,reloading of data sets is not required.** Moreover,the used command history is saved automatically for later use.
- **R can be used both in interactive mode for immediate outputs or on demand results through interaction with the user as well as in batch mode for a set of outputs at the end of execution or for periodic run.**

The Basic Features of R Programming Language

We know that **any language is a set of characters, words and syntactic rules used to express ideas for a meaningful communication**.There is no exception to this for the R language also.So, we shall study R from this perspective unlike other text books on R. We shall see in steps that R is a very simple,yet beautiful as well as highly powerful computer programming language.we shall also observe that it can be used like other high level programming languages for developing user-friendly applications of all types. We shall,therefore, study the language both as a means of general purpose program development as well as a platform for specialised arenas of applications like Data Science.

Since every language be it a natural human language or a computer programming language must have a definite set of symbols called the **Character Set** of the language,it is true for R also.So,we need to learn that first.

The R character Set

The term **character set** implies the set of symbols used in a language or a system of writing or printing.

The R character set comprises 88 characters as mentioned below.

The R character Set

Characters	Name	Count
A-Z, a-z	Letters of the English Alphabet	52
0-9	Digits	10
, ; .( ){ }[ ] _ % /+-* <>|: = & ^! # $ space	Special characters	26
Total:		88

It may be noted that we can use both uppercase and lowercase letters of the English alphabet for defining something in R.However,**R is case-sensitive**.This implies that anything defined by using,say, uppercase letters can never be referenced later by using its lowercase form.

Words in R

Being a language,R must consist of words.A word of a language may be defined as a single distinct meaningful element of speech or writing.An R word is defined as a consecutive set of characters from its character set that satisfies the following rules:

Words in R...the extended concept

- It can be a combination of **letters, digits, period (.) and underscore (_)**
- **It must start with a letter or a period**. **If it starts with a period, it cannot be followed by a digit.**
- **It can be of any length effectively**

Like every other language,R comprises two kinds of words, namely,

- **Reserved Words** and
- **User-defined Words**

The **Reserved Words** of a programming language are the words that act like the building blocks of the language each having a definite application with a particular **syntax and semantics.** By the term **syntax**, we mean the set of rules that defines the arrangement of words or symbols in a language.on the other hand,the term **Semantics** means the meaning implied by words, phrases and sentences of a language.For instance,**the rock smelled the color five** is a syntactically correct sentence but it has absolutely no meaning because rocks do not smell and, even if they could, they couldn't smell a color that doesn't exist.

Continued concepts of Words in R

Reserved Words of natural languages are the words found in the dictionaries of the languages.Most of the reserved words in R have been formed with lowercase letters of the character set.The reserved words cannot be used for any other purposes except for the purposes for which they have been defined.
The **User-defined Words** are the products of human mind which are used to assign a name to any user-defined entity to be used in the R environment.Some examples of user-defined words may be (i)name a person,(ii)name a memory location in the computer etc.

Variables and identifiers in R

A **variable** is a named location in the main memory of a computer being used which is defined for storing data according to the need of a user.As the value stored in such a location may change (i.e. vary),it is popularly known as a **variable**.Variables having unique names are known as **identifiers**, since they identify a specific location in the main memory without any ambiguity. Thus,the combination of characters **salary**, **Amount.Paid**, **.value** are valid identifiers; whereas, **TRUE**(being a reserved word), **.1salary**(Violates the second rule), **_name**(Violates the second rule), **123**(Violates the second rule) etc. are invalid variables or identifiers.

Compound User-defined words and R constants

Best Practices to form compound user-defined words

Since, the earlier versions of R used underscore (_) as an assignment operator hence,the period (.) was used extensively in variable names having multiple words. Current versions of R support underscore as a valid character in an identifier but it is good practice to use period as word separators to avoid confusion in using programs in earlier versions.

For example, a.variable.name is preferred over a_variable_name or alternatively we could use camel case in a variable name such as MonthlySalary.

R constants

R Constants are the entities the values of which remain unaltered under all circumstances. **R constants** are basically of two types, namely, **system-defined constants** and **user-defined constants**. **The system-defined constants are some reserved R words** denoting some frequently required values, such as the value of the mathematical constant π. The value of this constant can be obtained by writing pi.

Similarly, we use the word **LETTERS** to have the letters of the English alphabet in uppercase form.

R constants Contd.

The word **letters** is used to have the letters of the English alphabet in lowercase form, **month.name** to have the names of the Julian months in full forms and **month.abb** to have the names of the months in abbreviated form as seen in most of the calendars, are some of the many system-defined constants. So, the system-defined constants can be thought of as the constants built into the system.

The **user-defined constants** are the ones which the users may require to define to serve some specific purpose such as writing a formula, creating a prompt for the user etc. These are basically of two kinds, namely, **numeric constants** and **character constants**. A **numeric constant** is any number consisting of any numeric data. whereas, a **character constant** is any sequence of characters written by the user enclosed within double or single quotation signs. So, in the formula: 2*pi*r, the number 2 is a numeric constant, pi is a system-defined constant and r is an identifier containing some value entered by the user; and "WELCOME To R Programming", 'Enter any whole number', "AS!@$fghj??))12345"" etc. are character constants.

R Operators

An **operator in R** is a special symbol or a collection of special symbols that signifies a task to be performed on values specified through variables, constants or any expressions and tells the computer to perform that task when used by following the grammatical rule of the language. The task is called an operation and the values on which the operation is performed are called operands. If an operator operates on a single operand, then it is called a **unary operator**, if it operates on two operands at a time then it is called a **binary operator**, if it operates on three operands at a time then it is called a **ternary operator** and so on. The operations can be **assignment,arithmetic, logical or bitwise logical**. Depending on the nature of operations, the operators can be classified into the following categories:

1. **Arithmetic operators**
2. **Relational operators**
3. **Logical operators** and
4. **Assignment operators**

Concepts on R Operators

R Arithmetic Operators are used to accomplish arithmetic operations on two R operands of same data type. Assuming that the identifiers **a** and **b** are of same data type, we illustrate the use of the arithmetic operators and their functionalities in the following the following table:

Operator	Description	Usage
+	Addition of two operands	a + b
–	Subtraction of second operand from first	a – b
*	Multiplication of two operands	a * b
/	Division of first operand with second	a / b
%%	Remainder of 1st operand divided by the second	a %% b
%/%	Integer quotient of the division	a %/% b
^	First operand raised to the power of the second	a^b

R Relational Operators are those that find out relationship between the two operands provided to them. Following are the six relational operations R programming language supports. The output is Boolean (**TRUE** or **FALSE**) for all of the Relational Operators in R programming language.

Concepts on R Operators–Contd.

Operator	Description	Usage
<	Is first operand less than second operand?	a <b
>	Is first operand greater than second operand?	a >b
==	Is first operand equal to second operand?	a == b
<=	Is first operand less than or equal to second operand?	a <= b
>=	Is first operand greater than or equal to second operand?	a >= b
!=	Is first operand not equal to second operand?	a!=b

R Logical Operators work only for the basic data types **logical, numeric and complex and vectors**. The following table depicts the logical operators with illustration of its usage:

Operator	Description	Usage
&	Element wise **logical AND** operation	a & b
|	Element wise **logical OR** operation	a | b
!	Element wise **logical NOT** operation	
&&	Operand wise **logical AND** operation	a && b
||	Operand wise **logical OR** operation	a || b

Continued Concepts on R operators

Bitwise Logical Operators in R

Bitwise logical operators are used to perform logical operations bit by bit of the two numbers involved.The following table illustrates all such bitwise logical operations.

Operator	Description	Usage
bitwOr	bitwise OR	bitwOr(value1,value2)
bitwXor	bitwise XOR	bitwXor(value1,value2)
bitWnot	bitwise NOT	bitwNot(value)
bitwAnd	bitwise AND	bitwAnd(val1,val2)
bitwShiftL	Left shift	bitwShiftL(shiftvalue,value)
bitwShiftR	Right shift	bitwShiftR(shift,value)

R Assignment Operators are those that help in assigning (i.e. storing) a value to a variable or an identifier. The following table depicts the assignment allowed operators with illustration of its usage:

Operator	Description	Usage
=	Assigns right side value to left side operand	a = 5
<-	Assigns right side value to left side operand	a <- 7
->	Assigns left side value to right side operand	9->a
<<-	Assigns right side value to left side operand	a <<- 9.5
->>	Assigns left side value to right side operand	1..5 >>a

Precedence and associativity of different operators in R

The term **Precedence** defines the order of execution, i.e., which operator gets higher priority compared with the other associated operators during execution in R environment.For example, in the expression: **a-b*c**, the multiplication operator gets higher priority compared to the subtraction operator;hence, the multiplication operation will precede the subtraction operation in the evaluation of the value of the expression.For this reason,we say that the **'*'** operator has a higher precedence then the **'-'** operator.
The term **associativity** is related to an operator. In programming languages, it implies a property that determines how operators of the same precedence are grouped in the absence of parentheses. In case an operand is both preceded and followed by operators (for example, "*a*"), and those operators are of equal precedence, then the operand may be used as input to two different operations (i.e. the two operations indicated by the two operators). The choice of operation applicable to the operand is determined by the "**associativity**" of the operators.

Precedence and associativity of different operators in R–Contd.

Operators may be associative (meaning the operations can be grouped arbitrarily, i.e. in any order), **left-associative** (meaning the operations are grouped from the left), **right-associative** (meaning the operations are grouped from the right) or **non-associative** (meaning operations cannot be chained, often because the output type is incompatible with the input types).

The **associativity and precedence** of an operator is a part of the definition of a programming language; different programming languages may have different associativity and precedence for the same type of operator.

Let us consider the expression **x**$\sim y \sim$**z**, where '$\sim$' denotes any recognized operator. If the operator $\sim$ has left associativity, this expression would be interpreted as **(x**$\sim$**y)**$\sim$**z**. If the operator has right associativity, the expression would be interpreted as **x** $\sim$ ($y \sim$ **z)**. If the operator is non-associative, the expression might be a syntax error, or it might have some special meaning. Some mathematical operators have inherent associativity. For example, **subtraction and division, as used in conventional mathematical notation, are inherently left-associative. Addition and multiplication, by contrast, are both left and right associative.** (e.g. x– y – z= (x – y) – z; but, (x-y)-z x-(y-z) except in few special cases).

The following table shows the **precedence and associativity of operators in R**:

Operator precedence in R

Operator	Description	Associativity
^	Exponential operator	Right to left; i.e. a^b=a^b
-x, +x	Unary minus, unary plus	Left to right
%%	Modulus or modulo operator	Left to right
*, /	Multiplication, division	Left to Right
+, -	Addition, subtraction	Left to right
<, >, ==,>=, <=, !=	Logical or Boolean operators for comparisons	Left to right
!	Logical NOT	Left to right
&, & &	Logical AND	Left to right
|, ||	Logical OR	Left to right
->, ->	Rightward assignment	Left to Right
<-, <-<-	Leftward assignment	Right to left
=	Leftward assignment	Right to left

The difference between **&** and **&&** should be noted carefully,The shorter one is used to perform **elementwise comparisons like arithmetic operators while the longer form evaluates from left to right examining only the first element of each vector**(to be introduced later). The evaluation proceeds only until the result is determined. The longer form is appropriate in control-flow statements , particulaly with **'if' clauses**.

Again, between | and ||, the shorter one is used to perform elementwise comparisons like arithmetic operators while the longer form evaluates from left to right examining only the first element of each vector(to be introduced later). The evaluation proceeds only until the result is determined. The longer form is appropriate in control-flow statements , particulaly with **'if' clauses**.

Installation For Getting R in a Working System

R is freely available from the **C**omprehensive **R** **A**rchive **N**etwork (**CRAN**) at **http://cran.r-project.org**.

Precompiled binary distributions of the base system and contributed packages are shown available for

- **Linux(Debian, Fedora/Redhat, Ubuntu)**
- **Mac OS X**, and
- **Windows**

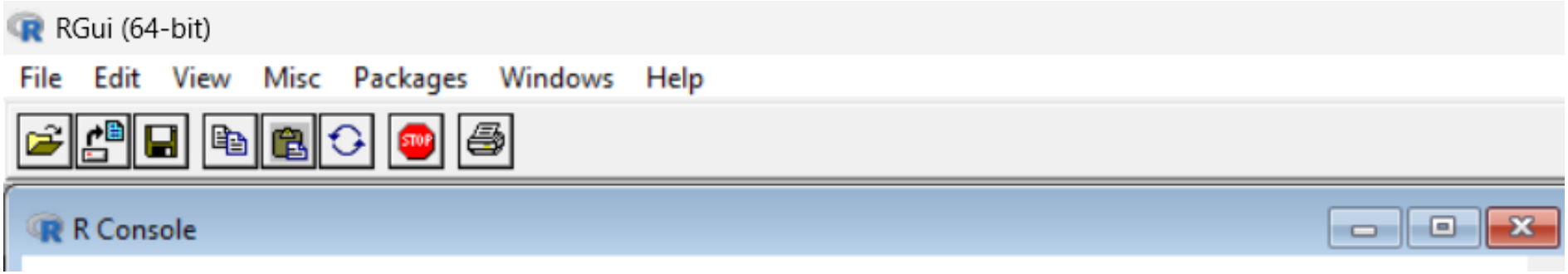
The desired link among the ones shown here is to be clicked for downloading the application environment.The directions for the installation of the base product on the target platform of the system are then followed to get the software in the actionable form.

Starting R in a Computer

On a **Linux or Mac system**, R is started **by typing R** on the command line in a terminal window. On a Windows system,R is started **by clicking the R icon** on the desktop.

The result is a greeting and the R prompt with the > sign.The screen display will be of following kind:

R in Interactive Mode



R version 4.3.1 (2023-06-16 ucrt) – "Beagle Scouts" Copyright (C) 2023 The R Foundation for Statistical Computing Platform: $x86_64 - w64 - mingw32/x64(64 - bit)$

.....

Type **'demo()'** for some demos, **'help()'** for on-line help, or **'help.start()'** for an HTML browser interface to help. Type **'q()'** to quit R.

.....

During startup - Warning message:
In sys.load.image(".RData", FALSE) : non-uniform 'Rounding' sampler used
>

The window in which this screen appears is called the **R console**.

Working with R in interactive mode

We can now start working with R interactively. It is the best way to learn its syntax and semantics. We assume that the user possesses the basic operating concepts of a computer with the fundamentals of operating systems.

One can start R on Linux system by typing R at the Shell prompt. It shows the $ prompt where R runs in the background. On windows system, we start R by double-clicking on the R icon shown on the desktop. The prompt here is: '>'.

Statements in R

A Syntactically valid combination of R words and other characters forms an R statement.An R statement can also be called an **R command** which is basically an instruction for execution. An R instruction is meant for some desired action. We can type a command at the R prompt and press enter to get it executed.The R interpreter translates and executes the command immediately.In case of any error in the command,it shows error messages that can be understood easily. If desired, we can also change the default prompt of R with a command as illustrated below:

>**options (prompt = "R>")**

This shows the Prompt as under:

R> -

Interactive uses of R

If one likes to set the prompt to **an empty string**, it will result in error as shown below:

>**options (prompt="")**

Error in options (prompt=""): invalid value for 'prompt'.

If we like to save all our works in the R environment in a separate directory, then we create a new directory say, **"Rprograms"** , under the current working directory and then move to the new directory under the current working directory. We can check the current working directory by using the function **getwd()** as shown below:

>**getwd()** We press Enter after typing this.

[1] "C:/Users/Administrator/Documents" Will be the response of R or a similar one.

Next if we type the following and press **ENTER**,

>**setwd("Rprograms")**

The new working directory will be set to the desired one. We can check it by the return value of **getwd()** function as illustrated below:

>**getwd ()**

Interactive uses of R–CONTD.

The system will now show the following:

[1] "C:/Users/Administrator/Documents/Rprograms"

We can also go back to the previous level directory by typing the command:

>**setwd ("..")**

Assignment Statements in R

The term **"Assignment Statements in R"** implies the statents for storing some data value in an R identifier/variable.To store a value in a variable, we need an assignment operator. As stated earlier,the formal assignment operator in R is: <-, i.e. a left-arrow sign formed with less than operator followed by a hyphen. However, the sign of equality can also be used without any loss of generality as illustrated below:

X<-5 And X=5 , will imply the same.

However, an assignment can also be done by using the greater than sign and shall has to be done in the opposite direction as shown below:

>**5->x** is also okay. We can see the assigned value by writing just the variable name at the R prompt as shown below:

>**x**

[1] 5

Assignment Statements in R....Contd.

But, if we write,

5=x , then the following error message will be generated by the system:

Error in 5 = x: invalid (do_set) left-hand side to assignment

>**z=6**

>**z** # This is to display the content of z

[1] 6

>**t=TRUE** # A logical value is being assigned to the R identifier named as t

>**t**

[1] TRUE

>**v=2:9** # Here a sequence of values with a difference of 1 are being assigned to the R identifier named as v

>**v** # This is to display the contents of v

[1] 2 3 4 5 6 7 8 9

>**y="Welcome to R"** # some text is being assigned to an identifier y

Now, to see the assigned text, we write:

>**y** # This shows:

[1] "Welcome to R"

Let us now examine a special function in R which is very often required in assignment of data values in an R variable. It is **seq( ) function**.

The seq( ) function in R

The **seq( ) function** is used to generate a sequence of numbers. Let us first study its syntax:

seq(from = 1, to = 1, by = ((to - from)/(length.out - 1)),length.out = NULL, along.with = NULL, ...)

- **from, to: begin and end number of the sequence**
- **by: step, increment (Default is 1)**
- **length.out: length of the sequence.**

To generate a sequence from -6 to 7, for instance, we can write:

x=seq(from=-6,to=7)

To see the contents assigned to x, we enter:

>**x**

[1] -6 -5 -4 -3 -2 -1 0 1 2 3 4 5 6 7

>**x <- seq(-6,7,by=2)**

To see the contents assigned to x, we enter:>**x** # This shows the following values:

[1] -6 -4 -2 0 2 4 6

It may be noted that R can be used to play a number of roles as illustrated below.

R as a displayer

We can type a constant, assign a value to a variable at the R prompt and get it or the value implied by it displayed instantly. As we can save any previously created workspace in R, the variables defined earlier can also be displayed as illustrated below. Any error is pointed out by R immediately. In some cases, we explain something starting with a sign. This is nothing but a comment. Such comments can be written in a command line also without hampering the operation.

>**'A'+'B'**

Error in "A" + "B" : non-numeric argument to binary operator

>**3**

[1] 3

>**'s'**

[1] "s"

>**x**

[] 1 2 3 4 5 6 7 8 9 # This is a previously assigned set of values assigned with the statement: x=1:9

>**y**

Error: object 'y' not found,because 'y' was not defined earlier.

>**5**

[1] 5

R as a displayer

>**'It is R'**

[1] "It is R"

Let us now write:

>**month.names** # This is a built-in constant of R environment spelt wrongly that results into the following error: **Error: object 'month.names' not found**

>**month.name** # The above constant spelt rightly. The English names for the months of the year and hence it shows the following names:

[1] "January" "February" "March" "April" "May" "June"

[7] "July" "August" "September" "October" "November" "December"

>**exp(1)** This shows the value of mathematical e

[1] 2.718282

>**LETTERS** # This is built-in constant of R environment. The 26 upper-case letters of the Roman alphabet;

[1] "A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M" "N" "O" "P" "Q" "R" "S"

[20] "T" "U" "V" "W" "X" "Y" "Z"

>**letters** This is another built-in constant of R environment with a different output. The 26 lower-case letters of the Roman alphabet;

[1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m" "n" "o" "p" "q" "r" "s"

R as a displayer–CONTD.

[20] "t" "u" "v" "w" "x" "y" "z"
>**month.abb** #This is again a built-in constant of R environment. The three-letter abbreviations for the English month names
[1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"
>**pi** # A useful built-in constant of R environment. The ratio of the circumference of a circle to its diameter.
[1] 3.141593
Numbers preceded by 0X are recognized by R as hexadecimal numbers. If we type such a number in the R prompt then R shows the decimal equivalent of the number. For instance,
>**x20** # Trying to show the decimal value of a hexadecimal number in a wrong way.Error: object 'x20' not found
>**0x20** # This is the right way of finding out the decimal equivalent of hexadecimal 20.
[1] 32
>**oxafd** # Again the wrong way; letters must be in uppercase form.
Error: object 'oxafd' not found

R as a displayer–CONTD.

>2 # A numeric constant

[1] 2

>**as.roman(61)** # To see the equivalent Roman number equivalent to decimal 61

[1]LXI

>**as.numeric(as.roman("LXI"))** # To see the decimal equivalent of the Roman number **[61]**

R as a calculator

R code consists of one or more expressions. An expression is an R instruction to perform a specific task. **Consecutive instructions in R are separated either by a newline or by a semicolon**.

The R prompt can be treated as an R console and we can work on it just as we work on a calculator as illustrated below:

Two R statements written side by side:

>**5+3;5-3** The above statements return the following results:

[1] 8

[1] 2

R as a calculator Contd.

Let us now write a single statement:

>**23*13**

The above statement returns:

[1] 299

Let us check the division operation as follows:

>**67/13**

[1] 5.153846

For obtaining the remainder of a division, we write statements as shown below:

>**13%%2**

[1] 1, # We note that %% is a modulo operator that returns the remainder of integer division.

>**2^5**

[1] 32

Here, we note that caret symbol **(^)** is used for exponentiation i.e. to write the power of a base number.

R as a calculator Contd.

>67%/%13 # %/% implies integer division.

[1] 5 is the returned value.

There are many mathematical functions available in R. For example, to obtain the square root of a number, we can make use of **sqrt()** function; for logarithmic value to the base e, we can use **log()** function;for obtaining the factorial of a number we can make use of **factorial() function** etc.Following are some illustrative use of such functions:

>**sqrt(3)**

[1] 1.732051 is the returned value.

>**log(3)**

[1] 1.098612 is the returned value.

>**log(2)**

[1] 0.6931472 is the returned value.

>**log(1)**

[1] 0 is the returned value.

>**factorial(6)**

[1] 720 is the returned value.

Objects in R

Every entity in R with which we deal is called an Object. Thus, a variable or identifier that we create to work with is also an object. R supports 6 basic classes of objects as stated below:

1. **Character**
2. **Numeric (Real numbers)**
3. **Integer (whole numbers)**
4. **Complex**(Imaginary numbers)
5. **Logical (TRUE/FALSE)**
6. **Raw**

Objects of any of these classes are known as **atomic structure** meaning that components of that object are of same type or mode. The term mode implies the basic type of the fundamental constituents of the object. Objects of character class may contain any set of values enclosed within single or double quotes. So, such values cannot be used for arithmetic calculation. Objects of numeric class can be any number with or without decimal part. Objects of integer class can be obtained by storing any numeric value in an identifier using the **as.integer()** function as illustrated below:

Objects in R–Contd.

i=as.integer(5)

class(i)

[1] "integer"

But the mode of i will be numeric as shown below:

mode(i)

[1] "numeric"

Objects of complex class are created by storing a complex number in an identifier. We know that complex numbers are of the form: x+yi, where x and y are any real numbers including 0. Here are some illustrations:

z=2+3i

t= 5+oi

v= -1+oi etc.

An R object becomes a logical one if we store either T or TRUE or F or FALSE value in it as illustrated below:

x=TRUE

as.logical(x)

[1] TRUE

Objects in R–Contd.

The raw data type is meant for obtaining the hexadecimal value of a number as shown below:

```
>a=16
>as.raw(16)
[1] 10 # Hexadecimal value equivalent to decimal 16.
```

Apart from these data types, R supports another data type called double for integer arithmetic of higher precision. Numbers of double type are double precision numbers. The additional bits in storage not only increase the accuracy in the calculated value, but also the range of magnitude that can be represented. The use of double is illustrated below:

```
>s=double
>s=6
>s
[1] 6
>y=double (2)
>y
[1] 0 0
```

Objects in R–Contd.

R constants can be checked with **typeof()** function. Numbers are stored in double precision format by default. For integer constants, numbers must be followed by L and complex constants, the numbers must be followed by i. Use of the function is illustrated below:

```
>typeof(7)
[1] "double"
>typeof(7L)
[1] "integer"
>typeof(7i)
[1] "complex"
>typeof(LETTERS)
[1] "character"
>typeof("Ramdhanu")
[1] "character"
>typeof('12345')
[1] "character"
```

OUTCOME OF THIS LESSON

The outcome of going through this lesson is knowledge of the following concepts:

- **History of the R language**
- **Different platforms that support R**
- **The R character set**
- **Words, variables, identifiers and constants in R**
- **Operators in R**
- **Precedence and associativity in R**
- **Statements or Commands in R**
- **R as a Displayer**
- **R as a Desk calculator**
- **R as a Scripting Language for Data Manipulation**
- **Objects in R**